

Präsent-Produkte im Online-Shop

Technische Erklärung des Ist-Zustands und Blaupause für andere Webshops
Stand: 8. September 2026 · Bezugssystem: Shopify-Shop vinsecco-freund.de
und Toolkit-Repository Vinsecco

1. Zweck und Leserkreis

Dieses Papier fasst zusammen, wie die Produktkategorie **Präsente** (Geschenksets aus Wein, Feinkost und Zubehör) im Vinsecco-Shopify-Shop heute funktioniert, welche Daten wo liegen und warum Beschreibungstext und Nährwert-Reiter auseinanderlaufen können. Der zweite Teil beschreibt eine übertragbare Zielarchitektur für Shops, die Präsente neu einführen.

Adressaten sind Produktverantwortliche, Shop-Entwickler und Integratoren zwischen ERP und Storefront.

2. Kurzfassung

Ein Präsent ist ein **Elternartikel** (das Set, das der Kunde kauft) plus einer **Stückliste** aus **Kind-Artikeln** (Wein, Sauce, Gewürz, Schale). Der Nährwert-Reiter listet die strukturierten Stammdaten der Kinder nacheinander. Die fette Inhaltsliste auf der Produktdetailseite ist dagegen händisch gepflegter Freitext am Parent und wird nicht aus der Stückliste erzeugt.

Regel für die Zielarchitektur: Nährwerte und Legalangaben nie auf das Set kopieren. Single Source of Truth bleibt der Kind-Artikel. Der Parent speichert nur Verweise und Mengen.

3. Begriffe

Begriff	Bedeutung im Vinsecco-Kontext
---------	-------------------------------

Parent / Set	Verkaufbarer Präsent-Artikel, z. B. „Präsent African Spice-Box“, SKU 824006, Shopify product_type = Präsent.
Kind / Komponente	Eigenständiger Shop-Artikel (Wein, Feinkost, Zubehör) mit eigenen Metafields.
Stückliste (BOM)	ERP-Rezeptur: welches Kind in welcher Menge in welchem Parent steckt. Tabelle KHKArtikelStueckliste.
body_html	Shopify-Produktbeschreibung des Parents. Enthält Marketingtext und die fette Inhaltsliste.
Metafields	Strukturierte Felder am Kind, z. B. custom.jahrgang, custom.brennwert_kcal, custom.zutaten.

4. Ist-Zustand im Vinsecco-Shop

4.1 Beispiel African Spice-Box

Die Produktdetailseite teilt sich in zwei Schichten, die im Shop unabhängig befüllt werden:

Fläche	Was der Kunde sieht	Datenquelle
Hero / Beschreibung	Titel, Preis, Marketingtext, fette Liste der Karton-Inhalte, Warenkorb	Shopify-Produktfelder des Parents, vor allem body_html
Reiter „Präsente – Schenken mit Genuß“	Allgemeiner Text über Genussgeschenke	Statischer Theme-Block, identisch auf allen Präsent-Seiten

Reiter „Nährwertangaben, Herkunft, etc.“	Pro Kind ein Spezifikationsblock (Wein- oder Food-Vorlage)	Metafields der verknüpften Kind-Produkte
--	--	--

4.2 Die Inhaltsliste ist Freitext am Parent

Die Zeilen „1 Flasche Windswept Sauvignon Blanc (False Bay) – 0,75 l“ usw. sind **nicht** zur Laufzeit aus der Stückliste gebaut. Sie stehen als HTML in der Beschreibung des Parents, etwa:

Absatz 1: Marketingtext („Südafrika für Genießer ...“)

Absatz 2: Überschrift „Im naturfarbenen Präsentkarton.“

Absatz 3: Positionen fett, mit Zeilenumbruch: Menge + Gebinde + Name + Marke in Klammern + Füllmenge

Das Theme gibt `product.description` aus. Im Vinsecco-Toolkit gibt es keine Funktion, die diese Liste aus Kind-Artikeln erzeugt. Die vorhandene `body_html`-Erzeugung gilt nur für Wein und Food (Charakteristik plus Gastro-Empfehlung).

4.3 Die Nährwerte hängen an den Kindern

Der zweite Reiter iteriert über die Kind-Produkte und verwendet dieselbe Feldvorlage wie die Wein- bzw. Food-Produktdetailseite. Die Werte kommen aus Shopify-Metafields, die das Toolkit aus ERP/CSV auf die Kinder schreibt. Beispiel False Bay Slow Chenin Blanc (SKU 605091):

Anzeige im Shop	Shopify-Metafield	CSV-/ERP-Feld	Sync-Funktion
Jahrgang	<code>custom.jahrgang</code>	JJahrgang	<code>update_ingredient_m</code>
Alkohol in Vol%	<code>custom.alkohol_vol</code>	JAlkGehaltLabel	<code>update_ingredient_m</code>
Restzucker (g/L)	<code>custom.restzucker</code>	JZucker	<code>update_ingredient_m</code>
Säure (g/L)	<code>custom.saure</code>	JSaeure	<code>_update_metafield_g</code>

Brennwert kcal / kJ	custom.brennwert_kcal / _kj	USER_NWEnergiekcal / kj	update_nutrition_me
Kohlenhydrate / Zucker	custom.kohlenhydrate / davon_zucker	USER_NWKohlenhydrate / NWdavonZucker	update_nutrition_me
Zutaten	custom.zutaten	USER_JInhalt	update_ingredient_m
Anbaugebiet	custom.region	WRegion	update_basic_info_m
Qualitätsstufe	custom.qualitätsstufe	WQualitaetsstufe	_update_metafield_g
Rebsorten	custom.rebsorten	JRebsorten	_update_metafield_g
Hersteller	custom.hersteller	WName	update_basic_info_m
Inverkehrbringer	custom.in_verkehr_bringer	USER_InVerkehrBringer	_update_metafield_g
Herkunftsland	custom.herkunftsland	WLand	update_basic_info_m
Flascheninhalt	custom.flascheninhalt	USER_Flascheninhalt	update_basic_info_m

Mapping: Datei metafield_mapping.py. Schreiben:
shopify_api_client.py, Kernmethode _update_metafield_group. Kinder
ohne Nährwerte (z. B. Keramikschale) erscheinen im Reiter nicht.

4.4 Die Stückliste im ERP

Im Warenwirtschaftssystem (Sage/KHK, Mandant 2) ist ein Präsent ein
Stücklistenartikel. Die Tabelle KHKArtikelStueckliste speichert je Zeile:

Feld	Inhalt
Stueckliste	Artikelnummer des Parents (z. B. 824006)
Element	Artikelnummer des Kindes (Wein, Food, Zubehör)
Menge	Anzahl im Karton (im Shop oft als „1.0 x ...“ sichtbar)

Die Stückliste enthält selbst keine Nährwerte. Die hängen am Kind in KHKArtikel / KHKArtikelVarianten. Die Query SQL_CoreData_Sets_Complete explodiert die BOM: eine Zeile pro Kind, mit Menge plus Stammdaten des Kindes. Filter am Parent: Stuecklistentyp = 1, Verkaufsartikel, aktiv, Artikelgruppe 80100 / 880 bzw. 800.

Präsent-SKUs beginnen mit 8 und werden aus den Wein-Exports bewusst ausgeschlossen (LEFT(Artikelnummer,1) NOT IN ('7','8')). Laden der Daten: Create_BasicDataSets_Complete() (Eltern) und Create_CoreDataSets_Complete() (explodierte Stückliste). Beide Funktionen schreiben derzeit nicht nach Shopify.

4.5 Zwei Quellen, deshalb Drift

An der African Spice-Box ist der Bruch sichtbar:

Stelle	Quelle	Wein
Fette Inhaltsliste	Freitext body_html am Parent	Windswept Sauvignon Blanc (SKU 605090, im Shop Draft)
Nährwert-Reiter	Verknüpfte Kind-Produkte	Slow Chenin Blanc (SKU 605091, veröffentlicht, Metafields aus Wein.csv)

Zusätzlich existiert ein zweites Reiter-Muster: „Präsent Schmaus im Schneckentempo“ zeigt zusammengeklebten Legaltext aus der Stückliste („1.0 x False Bay Slow Chenin Blanc Zutatenverzeichnis ...“). Damit gibt es im Shop zwei Arten, Kind-Daten darzustellen. Das erhöht die Abweichungsgefahr.

Präsent-Eltern stehen nicht in `Wein.csv` / `Food.csv` und erscheinen in `shop_only_skus.txt`: sie existieren im Shop, laufen aber nicht durch den Metafield-Sync der Einzelartikel.

5. Systemgrenzen

Schicht	Was sie leistet	Was sie nicht leistet
ERP (WKF / KHK)	Parent, Stückliste, Kind-Stammdaten, Nährwerte, Legalangaben	Storefront-Rendering
Vinsecco-Toolkit	CSV-Export, Metafield-Mapping, Sync auf Wein/Food-Kinder	Parent anlegen, Referenzen setzen, Inhaltsliste erzeugen
Shopify Admin / Theme	Parent-Produkt, händische Beschreibung, Tabs, Loop über Kinder	ERP als führendes System ersetzen

Das Theme-Liquid liegt nicht im Vinsecco-Repository. Die Storefront-Aussagen stammen aus den Live-Produktseiten, u. a. `/products/prasent-african-spice-box` und den Kind-PDPs.

6. Blaupause für andere Webshops

Ziel: Präsente als erste Klasse im Sortiment einführen, ohne Nährwerte zu duplizieren und ohne Inhaltslisten von Hand zu pflegen. Die Blaupause ist shop-agnostisch; Shopify ist das Referenz-Backend, das Muster gilt ebenso für andere Shops mit Produkt-Metafeldern und Stücklisten.

6.1 Leitprinzipien

- **Parent verkauft, Kind erklärt.** Preis, Bestand des Sets und Marketing gehören zum Parent. Spezifikation, Allergene und Herkunft gehören zum Kind.
- **Eine maschinenlesbare Stückliste.** Freitext darf Marketing sein, nie die einzige Quelle für Inhalt und Legalität.
- **Referenzen statt Kopie.** Ändert sich der Jahrgang eines Weins, aktualisiert der bestehende Kind-Sync alle Sets, die darauf zeigen.
- **Zubehör ohne Nährwert weglassen.** Schale, Löffel, Karton erscheinen in der Inhaltsliste, nicht im Nährwert-Reiter.

- **Theme folgt einem Vertrag, nicht Einzelfällen.** `product_type = Präsent` (oder gleichwertig) schaltet Layout und Tabs.

6.2 Datenmodell

Objekt	Pflichtfelder	Hinweise
Parent-Produkt	SKU, Titel, Preis, Status, <code>product_type</code> Gift/Präsent, Marketingtext	Kein eigenes Nährwert-Metafield-Set
Komponentenverweis	<code>custom.components</code> als Liste von Produktreferenzen	Reihenfolge = Anzeigereihenfolge im Reiter
Mengensatz	<code>custom.component_qty</code> JSON: <code>sku</code> , <code>qty</code> , <code>unit</code> , <code>label</code> , <code>show_nutrition</code>	Erzeugt die Inhaltsliste („1 Flasche ... 0,75 l“)
Kind-Produkt	Bestehende Nutrition-/Herkunfts-/Allergen-Metafields	Unverändert zum Einzelverkauf; Set konsumiert sie nur

Vorschlag für `component_qty`:

```
[{"sku": "605091", "qty": 1, "unit": "Flasche", "label": "Slow Chenin Blanc (False Bay)", "size": "0,75 l", "show_nutrition": true}]
```

Die sichtbare Inhaltsliste wird aus diesem JSON gerendert oder beim Sync nach `body_html` geschrieben. Manuelles Nachpflegen der Liste entfällt.

6.3 Pipeline (ERP → Shop)

1. Stückliste aus dem ERP lesen (Parent, Element, Menge), nur aktive Verkaufssets.
2. Jedes Element im Zielshop per SKU auflösen. Fehlt ein Kind, Set nicht veröffentlichen und Fehler loggen.
3. Parent anlegen oder aktualisieren (Titel, Preis, Typ, Tags, Status).
4. Produktreferenzen und Mengensatz schreiben.
5. Inhaltsliste aus dem Mengensatz erzeugen (Theme-seitig oder als generiertes HTML).
6. Kind-Metafields weiter über den bestehenden Artikel-Sync pflegen. Der Set-Job fasst sie nicht an.

Für Vinsecco heißt das konkret: `Create_CoreDataSets_Complete()` an einen Shopify-Writer hängen, der Parent-Upsert und `list.product_reference` übernimmt.

6.4 Theme-Vertrag

Bedingung	Verhalten
<code>product_type</code> ist Präsent / Gift / Set	PDP mit Inhaltsliste und zwei Reitern
Reiter 1	Shop-weiter redaktioneller Text (Schenken, Anlass) – nicht produktspezifisch
Reiter 2	Loop über <code>custom.components</code> ; Wein-Partial oder Food-Partial je Kind-Typ
<code>show_nutrition = false</code>	Position nur in der Inhaltsliste, nicht im Reiter
Kind Draft oder fehlende Pflichtfelder	Parent nicht kaufbar oder Reiter mit Hinweis; kein stilles Vertauschen durch ein anderes Kind

Es darf nur **ein** Reiter-Muster geben: strukturierte Metafield-Blöcke der Kinder. Zusammengeklebter Legaltext aus der BOM ist höchstens Fallback für den Export, nicht für die Storefront.

6.5 Was portabel ist – und was nicht

Portabel (in jeden Shop übertragbar)	Shop-spezifisch
--------------------------------------	-----------------

Parent/Kind-Trennung, BOM, Produktreferenzen, Mengen-JSON, Theme-Loop, Kind-Metafields als Wahrheit	Feldnamen, gesetzliche Labels (LMIV, Weinrecht), Artikelgruppen 80100/880/800, statischer Reiter-1-Text, ERP-Tabellen KHK
---	---

Nicht portieren: Nährwerte auf das Set denormalisieren. Jahrgang, Restzucker und Zutaten laufen sonst sofort aus der Synchronität.

6.6 Einführung in einem Shop ohne Präsente

1. **Kind-Basis prüfen.** Wein- und Food-Artikel müssen bereits spezifikationsvollständig im Shop stehen (oder parallel angebunden werden).
2. **Metafield-Definitionen anlegen.** `custom.components` (Liste Produktreferenz) und `custom.component_qty` (JSON).
3. **Produkttyp und Collection.** Typ „Präsent“ plus automatische Collection über Typ oder Tag.
4. **Theme-Partials.** Inhaltsliste, Gift-Tab, Nährwert-Loop; dieselben Partials wie auf der Kind-PDP wiederverwenden.
5. **Ein Pilot-Set.** Ein Parent mit 3–5 bekannten Kindern, davon mindestens ein Artikel ohne Nährwert.
6. **Sync-Job.** BOM → Referenzen → generierte Liste. Danach erst weitere Sets.
7. **Qualitätstor.** Veröffentlichung nur, wenn alle referenzierten Kinder existieren, aktiv sind und die Pflicht-Metafields tragen.

6.7 Betriebsregeln

- Kind ersetzen (anderer Jahrgang, anderer Wein) nur über die Stückliste, nie durch Editieren der Beschreibung.
- Parent-Bestand aus der schwächsten Komponente ableiten oder bewusst als eigenes Lager führen – beides ist zulässig, muss aber festliegen.
- Rechtstexte (Allergene, Inverkehrbringer) immer vom Kind, nie vom Set-Freitext.
- Wenn ein Kind unveröffentlicht ist, darf der Reiter nicht still auf ein anderes Produkt derselben Marke ausweichen.

7. Empfohlene nächste Schritte bei Vinsecco

Lücke	Risiko	Maßnahme
Sets werden nicht aus der Stückliste nach Shopify geschrieben	Referenzen und Beschreibung laufen auseinander	Create_CoreDataSets_Complete() an Parent-Upsert und Produktreferenzen koppeln
Inhaltsliste ist Freitext	Beworbener Inhalt ungleich Legal-Reiter	Liste aus component_qty erzeugen
Zwei Reiter-Formate im Shop	Uneinheitliche Darstellung, doppelte Pflege	Nur den Komponenten-Loop behalten
Spice-Box: Windswept in der Liste, Chenin Blanc im Reiter	Kunden- und Kennzeichnungsfehler	Referenzen gegen die Stückliste abgleichen; Draft-Kinder nicht still ersetzen

8. Schlussfolgerung

Präsente sind im Vinsecco-Shop fachlich bereits Parent-plus-Kinder, technisch aber noch zwei Welten: eine redaktionelle Beschreibung und ein Metafield-Reiter auf Kind-Produkten. Die Stückliste im ERP ist die fehlende verbindende Quelle. Wer das Konzept auf andere Shops überträgt, sollte genau diese Verbindung zuerst bauen – Referenzen und Mengen maschinenlesbar, Nährwerte ausschließlich am Kind, Storefront als reiner Konsument.

Dokument erzeugt aus der Code- und Storefront-Analyse des Vinsecco-Repositories, 8. September 2026.